AI-EDGE REU Program Final Report

To: Dr. Hongbo Li
From: Zayd Krunz
Date: July 25, 2025

Final Report on "*Mastering the game of Go with deep neural networks and tree search*"

Part 1: Summary of REU Program lectures

Over the past eight weeks, the REU program has provided a comprehensive introduction into the foundational concepts and applications of artificial intelligence. The curriculum began with some core principles of machine learning, such as distinguishing between supervised, unsupervised, and reinforcement learning paradigms used by state-of-the-art research labs like Meta, OpenAI, and others. We explored classical algorithms, from linear and logistic regression to training a gesture classification model on a Google platform, which established a solid practical groundwork for the later lessons. These later lessons then transitioned to deep learning, focusing on the architecture of neural networks. We explored elementary linear algebra and the process of stochastic gradient descent, which was crucial for understanding how models learn. A significant portion was dedicated to concepts borrowed from statistics, including the Bernouli Distribution, conditional probability, Bayes's Rule, and regularization techniques like the SoftMax function, which are essential for building generalizable models that don't just overfit on the training data, but truly "learn" from it.

The final weeks synthesized this knowledge by exploring advanced topics. We covered generative models for images, as well as generative models for text (known as Large Language Models, or "LLMs"). Additionally, we explored wireless networking design to great depth, including the intersection between AI/ML and the wireless edge. This program has not only equipped me with technical skills but has also fundamentally shaped my perspective on the current state as well as future trajectory of artificial intelligence research and deep learning.

Part 2: Summary of Selected Paper

Silver, D., et al. (2016). *Mastering the game of Go with deep neural networks and tree search*. Nature, 529(7587), 484-489.

This paper from Google DeepMind introduces AlphaGo, a computer program that defeated a human professional Go player for the first time - a milestone that was

previously thought to be at least a decade away. The authors address the immense challenge of Go, which is characterized by an *incomprehensibly massive* search space (b≈250, d≈150) and the difficulty of evaluating board positions, rendering traditional AI search methods like alpha-beta pruning ineffective.

AlphaGo's success stems from a novel integration of deep neural networks with a Monte Carlo Tree Search (MCTS) algorithm. The core of this novel architecture consists of two main neural networks:

1. **Policy Network:** This network is trained to predict the next legal move. It takes the current board state as input and outputs a probability distribution over all legal moves (where each individual probability is a floating-point number between 0 and 1). This very effectively narrows the enormous breadth of the search space by focusing the MCTS on the most promising moves, mimicking human intuition and rendering an otherwise computationally impossible problem feasible.
2. **Value Network:** This network is trained to evaluate board positions. It takes the board state as input and outputs a single scalar value that estimates the probability of winning the entire game from that position. This reduces the need for the search to extend to the end of the game, effectively truncating the search depth by several orders of magnitude.

The training process for these networks is an integrated multi stage pipeline:

- **Supervised Learning (SL):** The policy network is first trained on a large database of expert human games (from the KGS Go Server). This allows it to learn to imitate high-level human play, achieving a strong initial performance.
- **Reinforcement Learning (RL):** The SL policy network is then refined through self-play. The network plays thousands of games against previous versions of itself and is rewarded for winning. This RL process tunes the network towards the objective of winning, rather than just predicting the most likely human move (which may not be a good move).
- **Value Network Training:** Finally, the value network is trained on the outcomes of the self-play games generated by the RL policy network, learning to accurately predict the winner from any given position.

AlphaGo combines these three components in its MCTS algorithm. During a search, the policy network guides move selection, and at the end of a search path (a “leaf node”), the position is evaluated by both the value network and a fast "rollout" policy (a simpler, faster version of the policy network that plays to the end of the game). These two evaluations are combined to produce quite a robust assessment of the position, which is then backed up through the search tree to

inform the best move from the root.

In it’s evaluation, a distributed version of AlphaGo achieved a 99.8% win rate against other Go programs and, most notably, defeated the European Go champion, Fan Hui, 5-0 in a formal match.

Part 3: Personal Understanding and Reflections

The AlphaGo paper is a landmark not just for its groundbreaking achievement in Go, but for the elegance of it’s solution, which feels less like a brute-force computation and more like a blueprint for tackling complex decision-making problems in general. My primary reflection is on the parallels between this approach and the challenges in quantitative finance, my intended career field.

The stock market, like Go, is a high-dimensional, highly stochastic environment with a massive "search space" of potential strategies. A direct, exhaustive search for an optimal trading strategy is impossible. AlphaGo's dual-network approach offers a compelling paradigm. One could envision a "policy network" for trading that, given market data, suggests a distribution of potential actions (“buy”, “sell”, “hold” and at what volume). A corresponding "value network" could then evaluate the "state" of a portfolio after a potential trade, estimating it’s long-term return (the win condition). The training pipeline is also highly relevant as one could train a model via supervised learning on the trades of successful portfolio managers, then refine it through reinforcement learning in a simulated market environment. The paper demonstrates that a system can learn powerful, non-obvious strategies that outperform even the best human experts.

From a computer science perspective, the paper is a masterclass in optimization. MCTS on its own was a significant step for Go, but it hit a performance plateau. Deep learning on its own is a powerful function approximator, but it lacks the foresight of a true search. The genius of AlphaGo is in using the neural networks to make the tree search "smarter." The policy network prunes the tree, and the value network cuts it short. This hybrid approach, where learning guides search, seems broadly applicable to other intractable problems, from protein folding to logistics optimization. It makes me question the traditional separation between search algorithms and machine learning in the competitive programming space and consider how they might be more deeply integrated.

However, a skeptical view is also warranted. While impressive, AlphaGo's intelligence is relatively narrow. It has mastered Go, but it cannot explain *why* a move is good in human terms, nor can it apply its learning to chess or checkers.

Furthermore, the computational resources required for training and execution (1,202 CPUs and 176 GPUs for the distributed version) are immense and not easily accessible to the average Go player. This raises questions about the scalability and accessibility of such methods for problems that don't have the backing of a major tech corporation like Google. As of now, this is clearly not a universally accessible technique.

Ultimately, this paper has solidified my desire to pursue research at the intersection of AI and complex systems. It shows that with the right combination of learning and search, we can create systems that exceed human capabilities in domains of immense complexity. It serves as both an inspiration and a challenge to not only understand these methods but to think critically about how they can be generalized, made more efficient, and applied effectively to solve real-world problems.